

EventHub

Test Technique : Gestion d'Événements & Inscriptions (PERN Stack)

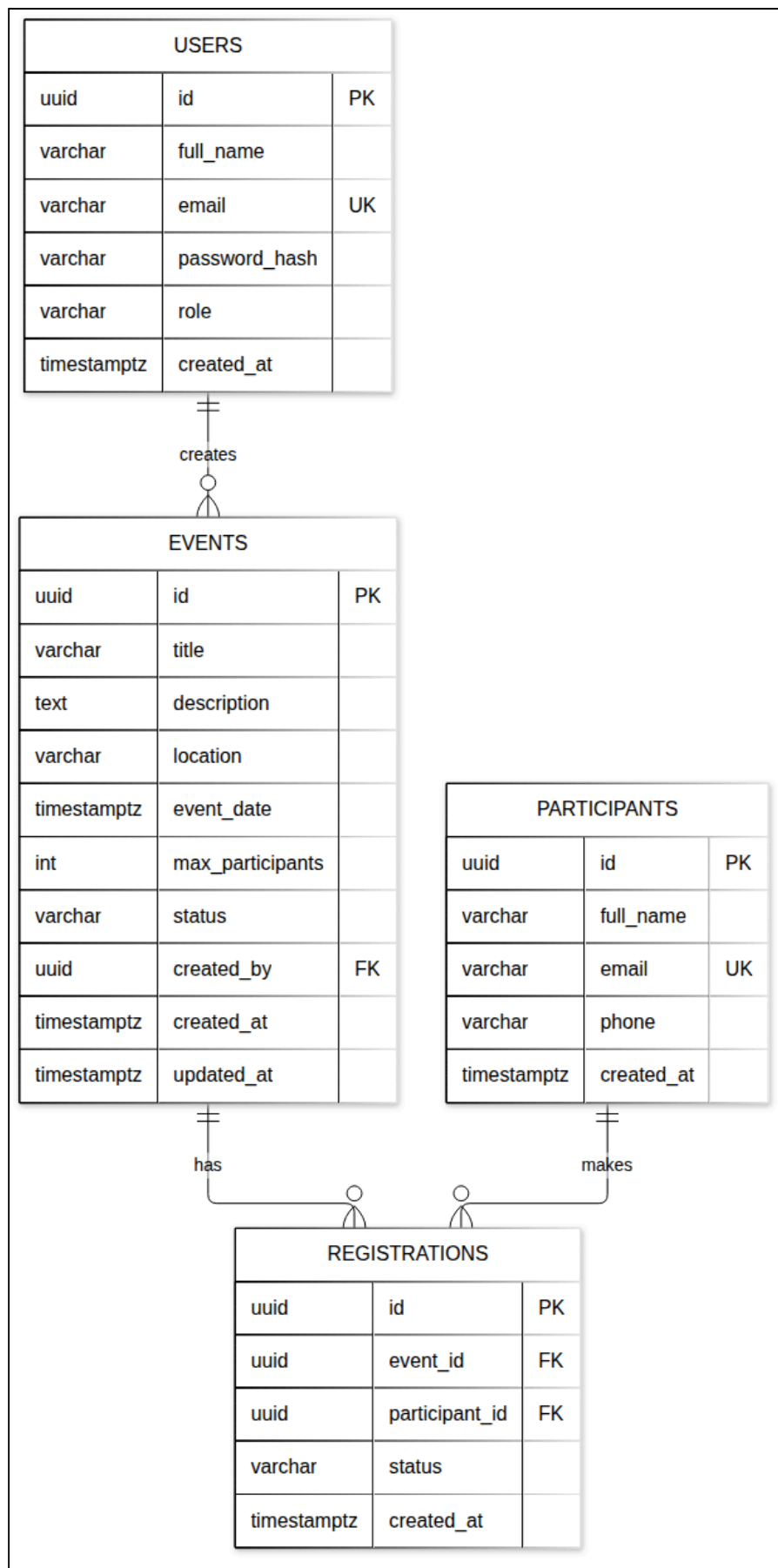
Document de conception

1. Présentation

EventHub est une application web permettant de gérer des **événements**, des **participants** et leurs **inscriptions**, avec un tableau de bord statistique. Les utilisateurs authentifiés créent et publient des événements, puis inscrivent des participants.

- **Base de données** : PostgreSQL (clés primaires UUID)
- **Backend** : Node.js + Express, authentification JWT, bcrypt, validation des entrées
- **Frontend** : React + Vite
- **Rôles** : `admin` (gestion complète : événements + utilisateurs) et `staff` (événements, participants, inscriptions)

2. Diagramme entité-relation (ERD)



Lecture du diagramme

Relation	Cardinalité	Signification
USERS → EVENTS	1 – N	Un utilisateur crée plusieurs événements ; chaque événement a un seul créateur (<code>created_by</code>).
EVENTS → REGISTRATIONS	1 – N	Un événement possède plusieurs inscriptions.
PARTICIPANTS → REGISTRATIONS	1 – N	Un participant peut avoir plusieurs inscriptions.
EVENTS ↔ PARTICIPANTS	N – N	Résolue par la table REGISTRATIONS , qui porte aussi le <code>status</code> de l'inscription.

3. Description des tables

users

Colonne	Type	Contraintes
id	uuid	PK
full_name	varchar	NOT NULL
email	varchar	NOT NULL, UNIQUE
password_hash	varchar	NOT NULL (haché avec bcrypt)
role	varchar	NOT NULL, CHECK dans (<code>admin</code> , <code>staff</code>)
created_at	timestampz	NOT NULL, défaut now()

events

Colonne	Type	Contraintes
id	uuid	PK
title	varchar	NOT NULL
description	text	facultatif
location	varchar	NOT NULL
event_date	timestampz	NOT NULL (date + heure)
max_participants	int	NOT NULL, CHECK > 0
status	varchar	NOT NULL, CHECK dans (<code>draft</code> , <code>published</code> , <code>cancelled</code>), défaut <code>draft</code>
created_by	uuid	FK → users(id), NOT NULL
created_at	timestampz	NOT NULL, défaut now()
updated_at	timestampz	NOT NULL, défaut now()

participants

Colonne	Type	Contraintes
id	uuid	PK
full_name	varchar	NOT NULL
email	varchar	NOT NULL, UNIQUE
phone	varchar	facultatif
created_at	timestampz	NOT NULL, défaut now()

registrations

Colonne	Type	Contraintes
id	uuid	PK
event_id	uuid	FK → events(id), NOT NULL
participant_id	uuid	FK → participants(id), NOT NULL
status	varchar	NOT NULL, CHECK dans (<code>pending</code> , <code>confirmed</code> , <code>cancelled</code>), défaut <code>pending</code>
created_at	timestampz	NOT NULL, défaut now()
UNIQUE (event_id, participant_id) : une seule inscription par participant et par événement		

4. Contraintes importantes

- `participants.email` et `users.email` sont **UNIQUE**.
- `registrations (event_id, participant_id)` est **UNIQUE**.
- Contraintes **CHECK** sur `role`, `events.status`, `registrations.status` et `max_participants > 0`.
- Toutes les clés étrangères sont **NOT NULL**. La suppression d'un utilisateur ayant créé des événements est bloquée, et les événements sont annulés plutôt que supprimés.

5. Index recommandés

Index	Objectif
<code>events(status)</code>	Filtrer les événements par statut
<code>events(event_date)</code>	Filtrer / trier par date
Index trigramme sur <code>participants(full_name, email)</code>	Recherche rapide par nom ou email
<code>registrations(event_id, status)</code>	Compter les inscriptions d'un événement, liste dans le détail
<code>registrations(participant_id)</code>	Retrouver les inscriptions d'un participant
<code>registrations(created_at)</code>	Dashboard : inscriptions du jour

Les contraintes UNIQUE créent déjà des index sur les emails et sur `(event_id, participant_id)`.

6. Règles métier

#	Règle	Mise en œuvre
1	Impossible d'inscrire un participant sur un événement non publié	L'API vérifie <code>status = 'published'</code> avant l'insertion
2	Un participant ne peut pas s'inscrire 2 fois au même événement	Contrainte UNIQUE (event_id, participant_id) en base
3	Ne pas dépasser <code>maxParticipants</code>	Transaction : verrouillage de la ligne de l'événement, comptage des inscriptions non annulées, blocage si complet
4	Annuler un événement annule toutes ses inscriptions	Une seule transaction met à jour l'événement et ses inscriptions

Remarque : seules les inscriptions `pending` et `confirmed` comptent dans la capacité. Une inscription annulée libère sa place. Un participant qui s'était désinscrit est réinscrit en réactivant sa ligne existante, avec les mêmes vérifications (règles 1 et 3).

7. Cycle de vie des statuts

- **Événement** : `draft` → `published` → `cancelled` (un brouillon peut aussi être annulé ; `cancelled` est final).
- **Inscription** : `pending` → `confirmed` → `cancelled` (une inscription en attente peut aussi être annulée).

8. API REST et sécurité

Ressource	Endpoints
Auth	POST /api/auth/login · GET /api/auth/me
Events	POST /api/events · GET /api/events?status=&date= · GET /api/events/:id · PUT /api/events/:id · PATCH /api/events/:id/status
Participants	POST /api/participants · GET /api/participants?search= · PUT /api/participants/:id
Registrations	POST /api/registrations · GET /api/registrations?eventId=&status= · PATCH /api/registrations/:id/status

Toutes les routes (sauf le login) exigent un JWT valide. Codes d'erreur : **400** validation, **401** non authentifié, **403** rôle insuffisant, **404** introuvable, **409** conflit (doublon / complet), **500** erreur serveur.

9. Données du dashboard

- Nombre total d'événements
- Nombre d'événements publiés
- Inscriptions du jour
- Top 5 des événements les plus remplis (inscriptions ÷ max_participants)